Round 1 prototype: one payment through the code

Alice pays v to Bob. Every arrow is a CBOR frame named after its `tv_core::wire` type; every box names the function that runs. Crates: `tv-cli` (`tv_client`), `tv-bank`, `tv-enclave`, `tv_core`.

How to read. Time flows down. Lanes are processes; the lane between Alice and Bob holds the files they hand each other (out of band, no network). blue sealed to the attested channel key, readable only in the enclave · bank check run by the bank on its own · enclave check that rests on the enclave · P2 property the step carries · key key used · file persisted (written atomically: temporary file, fsync, rename, directory fsync) · $^+$ next value: $pk^+ = pk_{i+1}$, $r^+ = r_{i+1}$ · m is the pay digest

Bank's machine (one host; the enclave is reachable only through tv-bank)

Alice: payer — `tv-cli pay, finish (tv_client)`

Files handed over — out of band, user side

Bob: payee — `tv-cli request, receive`

tv-bank: front — `server::handle_connection, Bank::handle_from, quota`

tv-enclave — `Enclave::handle, ChannelKeys, checks`

tv-bank: store — `Store, Committer, data dir D/`

1 Bob asks for a payment and gets a fresh channel key

`tv-cli request --amount v --out request.json`
`ops::Session::request → payment::request`: refuses while on the enrolled pk_0 (`EnrolledSerial`);
`request_commitment`: $v > 0$, $b_B + v \le B_{max}$, $C_B = \mathrm{Com}(b_B + v,\ pk_B^+,\ r_B^+)$
`BankClient::attest(pk_B)`: $nonce_B$ = 32 random bytes

❶ `BankRequest::Attest` $\{nonce_B,\ owner{=}pk_B\}$
TCP, one request per connection, `BankClient::call` retries with backoff

`handle_connection`: frame of at most 64 KiB, `source_of(peer)` = host address (memory only) →
`Bank::handle_from` → `bank::attest::attest`

❷ `EnclaveRequest::Attest` $\{nonce_B,\ owner{=}\mathrm{Some}(pk_B)\}$
`TcpLink` (vsock on Nitro), bounded by
`ENCLAVE_CALL_TIMEOUT` 10 s; no client address is sent

`Enclave::attest`
`ChannelKeys::issue(now, owner)`: fresh X25519 `EnclaveKeypair` (channel key); table entry (channel pk ↔ keypair, created, owner pk_B); evicts keys older than `MAX_KEY_AGE` (10 min) and the oldest beyond `MAX_PENDING_KEYS` (65536)
`provider.attest`: document over $nonce_B$, channel pk, `user_data` = `PROTOCOL_TAG` ‖ manifest digest (NSM on Nitro, mock signer with `--mock`)

❸ `EnclaveResponse::Attestation` $\{doc,\ manifest,\ channel_pk\}$

`manifest.bank == self.keys()`, else `Refused`
`AttestQuota::record(now, source, channel pk)`: per address at most `MAX_PENDING_KEYS_PER_SOURCE` (256) live keys; returns that address's oldest keys beyond the cap (availability)

only when over the cap: `EnclaveRequest::Cancel`
$\{channel_pk\}$ → `ChannelKeys::take` → `Cancelled`

❹ `BankResponse::Attestation` $\{doc,\ manifest\}$
(the channel pk travels inside the signed document)

`attest::verify_bundle`: `AttestationVerifier::verify` with $nonce_B$, prefix `PROTOCOL_TAG`, recomputed `manifest_digest`, PCR0 pin (`--pcr0`) or `--allow-mock` on loopback only; `check_fresh`: `timestamp` younger than `MAX_KEY_AGE`, at most `ATTEST_CLOCK_SKEW` (30 s) ahead; `flows::check_manifest`: bank keys = wallet's saves `wallet.json` `Pending::Request` $\{v,\ C_B,\ channel_pk,\ result_pk\}$ (after the attestation, since it holds its keys)

❺ `write_json`: `RequestFile`

`request.json` (payee controlled)
$\{v,\ pk_B,\ com_b_next{=}C_B^+,$
$attestation\{nonce_B,\ doc,\ manifest\}\}$

2 Alice checks the attestation offline and signs; she never contacts the bank's machine

❻ `read_json`: `tv-cli pay`
`--request request.json --amount v`

`ops::Session::pay_offer_with → payment::prepare_pay`
`verify_bundle` on the same document with Bob's $nonce_B$: refuses a stale (`StaleAttestation`) or future one; the payer's own freshness check
`manifest.bank` = her bank keys; request v = the agreed `--amount` (`PayAmount`, before anything is written); $v > 0$
`payer_commitment`: $C_A = \mathrm{Com}(b_A - v,\ pk_A^+,\ r_A^+)$ (`Insufficient` if $v > b_A$); refuses `SelfPayment`
m = `pay_digest`(pk_A, C_A, pk_B, C_B), sig_A = `sign_digest`(sk_A, m) (sk_a)
`bank::seal_to`(channel pk, `PaySecret` $(v,$ Opening $(b_A,\ r_A,\ \sigma_A,\ side,\ pk_A^+,\ r_A^+))$): `pt_tee::ClientChannel`, ephemeral X25519 (client key) → HKDF → ChaCha20-Poly1305; → `SealedBlob`$(channel_pk,\ nonce,\ ciphertext)$
`commit_pay`: saves `wallet.json` `Pending::Pay`(v, pk_B, C_B, C_A, m) before the offer file exists

❼ `write_json`: `OfferFile`

`offer.json`
$\{channel_pk,\ pk_A,\ C_A,\ sig_A,$ *payer* blob,
$pk_B,\ C_B\}$ (Bob cannot open the blob)

3 Bob submits; the bank checks what it can see, the enclave checks what is sealed

❽ `read_json`: `tv-cli receive --offer offer.json`

`payment::prepare_receive`: `check_offer`: pk_B, C_B match the pending request (`IOfferForOtherRequest`), same channel pk (`IOfferForEarlierRequest`)
m = `pay_digest`, sig_B = `sign_digest`(sk_B, m) (sk_b)
`seal_to`(channel pk, `PaySecret` $(v,$ Bob's Opening$)$) → payee blob

❾ `BankRequest::Receive` $\{channel_pk,$
`public:` `PayPublic` $\{pk_A, C_A, sig_A, pk_B, C_B, sig_B\},$
`payer: SealedBlob, payee: SealedBlob`$\}$

`bank::handlers::receive`
m = `pay_digest`(public part), recomputed by the bank
B1 `Bank::claim`(m): `stored_answer`: `Store::answer`(m) → if settled, return it once durable (bank) (P5);
`InFlight::try_claim`(m): one operation per digest, a copy waits on a `watch` and re-runs B1
B2 `verify_digest`(pk_A, m, sig_A), `verify_digest`(pk_B, m, sig_B) (bank) (P1) (P3)
B4a $pk_A \neq pk_B$, `serials_unspent`: `Index::is_spent` (bank) (P2)
`Bank::checked`: n = `random_bytes32` (bank nonce)

B1 `answer(m)`, B4a `is_spent` (store lock)

in-memory `Index` replayed from `log.jsonl`: spent set, enrolled set, commitment set, serial index, digest index (answers)

❿ `EnclaveRequest::Check` $\{bank_nonce{=}n,$
`op: CheckOp::Pay(Receive)`$\}$ (the whole `Receive`, blobs still sealed)

`Enclave::check` → `check_unscrubbed`
`ChannelKeys::peek`(channel pk): unknown or expired → `Refused`; `key.owner` $\neq$ `op.owner()` (pk_B) → `Refused` without consuming it
`open_ac`: payer blob, then payee blob, under the (channel key) (`Refusal::Opening`); then `consume`: `ChannelKeys::take` erases the key: one `Check` per key
`checks::check_pay`, all evaluated, `first_failure` picks the refusal:
`SameSerial` $pk_A^+ \neq pk_B^+$; `NextSerial`: $pk_A^+ \neq pk_B^+$, neither equals pk_A or pk_B
`Amount`: payer v = payee v
E1 `openings_are_signed`: $\mathrm{Com}(b, pk, r)$ with the public serial, σ under the (state key) (side `User`) or (issuance key) (side `Bank`) (enclave) (P2)
E3 `owners_signed`: sig_A, sig_B on m (repeats B2) (bank)
E4 `within_bounds`: $0 < v \le b_A$, $b_B + v \le B_{max}$ unless Bob's side is `Bank`, checked add (enclave) (P2)
E2 `NextCommitment`: $C_A^+ = \mathrm{Com}(b_A - v, pk_A^+, r_A^+)$, $C_B^+ = \mathrm{Com}(b_B + v, pk_B^+, r_B^+)$ (enclave) (P2)
`sign_result`(result key, m, sides) over `"tv/v1/result/ok"` ‖ n ‖ m ‖ sides. On refusal a result is still signed and dropped (`black_box`): same time whichever check failed (P4)
`scrub_stack` zeroes 64 KiB of dead stack; the binary runs on a zeroizing allocator (P4)

⓫ `EnclaveResponse::Result` (`EnclaveResult` $\{bank_nonce{=}n,\ digest{=}m,$
$sides{=}[side_A, side_B],\ sig\}$) or `Refused` (no reason)

4 The bank verifies the result, burns both serials, signs, and makes it durable before answering

`quota.consumed`(channel pk) frees Bob's slot
B3 `verify_result_with`(registered result key from `enclave.pk`, result, own n, own m) (enclave) (P2); on mismatch `re_register` attests the enclave again under the `serve` policy, then verifies once more
`Bank::settle`: `sign_state`(`signer(side)`, C') over `"tv/v1/state"` ‖ C' with the (state key) (user side) or (issuance key) (bank side) → σ'_A, σ'_B

⓬ `Bank::stage` under the store lock: **B4b** `answer(m)` and the spent set again (bank) (P2), then `Store::stage`$(\mathrm{LogEntry}\{seq,\ kind{=}Pay,\ serials{=}[pk_A, pk_B],$
$commitments{=}[C_A', C_B'],\ sigs{=}[\sigma'_A, \sigma'_B],\ digest{=}m\})$: applied to the `Index` at once, line buffered, returns a `Ticket`

`Committer::commit(ticket)`: **group commit**
first waiter takes the lead; `Store::take_flight`: every staged line plus a trailer `{"flight":{"len":N,"sha256":"..."}}`
`flush`: `spawn_blocking` one append and one **fsync** of `log.jsonl`, in a detached task
`confirm`: durable length advances; the leader drops the lead, then publishes on a `watch`; later stages join the next flight (P5)
on a write or fsync error: `roll_back` truncates to the durable length, rebuilds the `Index`, marks the flight's tickets `RolledBack` → `Unavailable`

⓭ `Durability::Durable` (ticket inside the durable prefix); the answer is stored under m for B1

⓮ `BankResponse::Settled` $\{sig_a_next{=}\sigma'_A,\ sig_b_next{=}\sigma'_B\}$
or `Refused` (uniform) or `Unavailable` (link down, rolled back)

5 Both wallets move to their next state

`submit_receive`: `verify_new_state`(C'_B, σ'_B) under the (state key); on `Refused` it attests again to tell `EnclaveRestarted` from `ReceiveRefused`
writes the receipt first, then `commit_receive`: `Wallet::advance`: index $i{+}1$, balance $b_B + v$, saves `wallet.json`

⓯ `write_json`: `ReceiptFile`

`receipt.json`
$\{digest{=}m,\ pk_A,\ C'_A,\ sig_a_next{=}\sigma'_A\}$

⓰ `tv-cli finish --receipt receipt.json` (no bank call)

or, without a receipt: `BankRequest::Lookup`$(serial{=}pk_A)$ → `BankResponse::Entry`(`LogEntry`) (`flows::lookup_sig`); the payer's only bank contact, it ties her address to pk_A

`flows::finish`: receipt digest = pending m and its C'_A = pending C'_A (`Mismatch`); `verify_new_state`(C'_A, σ'_A) under the (state key);
`Wallet::advance`: index $i{+}1$, balance $b_A - v$, `wallet.json` without `Pending`

Already done before this payment (second diagram): `tv-bank init --reserve N` wrote `state.key` `state key` `issuance.key` `issuance key` `bank-wallet.json` `bank-genesis.json` and `log.jsonl` holding only its header `{"log":"flights"}`. `tv-enclave --mock` booted with a fresh (result key) (`SigningKey::generate`, memory only) and the manifest (result pk, state pk, issuance pk); `register-enclave` (loopback only) ran `registry::attest_enclave` and wrote the result public key to `enclave.pk`. Both wallets are enrolled and moved off pk_0; Alice holds b_A on $pk_A = pk_1$, Bob b_B on $pk_B = pk_1$, each with (C, σ) signed by the `state key`.

Wallet keys (`tv_core::derive`, never sent): `seed` 32 random bytes in `wallet.json`;
sk_i = HKDF-SHA256(seed, salt "tv/v1", "sk"‖i), pk_i = Ed25519(sk_i), r_i = HKDF(…,"r"‖i), i as u64 BE.
A state is (b, pk_i, r_i); spending it moves the wallet to index $i{+}1$, so every serial is a one-time key.
`wallet.json` = seed, bank keys, index i, signed state, at most one `Pending`; one file lock per wallet.

Keys, who holds them, what they sign
`seed` wallet only — sk_i: owner one-time Ed25519 key, signs digests; r_i commitment randomness
client key ephemeral X25519 per `seal_to`, discarded after sealing
channel key X25519, one per `Attest`, in `ChannelKeys`; bound to its owner serial, single use, 10 min, enclave memory only
result key Ed25519, per enclave boot, memory only; public half in the manifest and in `enclave.pk`
state key `state.key`: signs user states Com (bank's key)
issuance key `issuance.key`: signs the bank wallet's states (total supply = reserve)

Primitives (`tv_core`), **every hash domain separated**
$\mathrm{Com}(b, pk, r)$ = SHA256("tv/v1/com" ‖ $b_{64\ BE}$ ‖ pk ‖ r): hiding by r, binding by collisions
state signature $\sigma = \mathrm{Sign}(k,$ "tv/v1/state" ‖ $C$$),$ k = state or issuance key
m_{pay} = SHA256("tv/v1/pay" ‖ pk_A ‖ C_A ‖ pk_B ‖ $C_B$$)$
$m_{refresh}$ = SHA256("tv/v1/refresh" ‖ pk ‖ $C$$)$, m_{enroll} = SHA256("tv/v1/enroll" ‖ pk_0 ‖ $C_0$$)$
result = Sign(result key, "tv/v1/result/ok" ‖ n ‖ m ‖ $sides$)
manifest digest = SHA256("tv/v1/manifest" ‖ $pk_{enclave}$ ‖ pk_{state} ‖ $pk_{issuance}$) in `user_data`
v is in no digest, so the bank recomputes m without learning it. $B_{max} = 10^9$ (`tv_core::B_MAX`)

Which check carries which property
P1 ownership: B2 (bank) (`handlers.rs`) before any enclave call; E3 repeats it. Holds with every enclave key leaked
P2 balance: E1, E2, E4 (enclave) (`checks.rs`); B3 (enclave) result over the bank's own n and m (`Bank::checked`); B4a (bank) before, B4b (store) under the store lock (`Bank::stage`): each serial spent once
P3 no fund destruction: B2 (bank), a burn needs the owner's signature
P4 privacy toward the bank: sealed blobs, v outside m, the old C_A only inside the blob, uniform `Refused`, constant time refusal, enclave log says only "check refused"
P5 crash safety: B1 stored answer (bank), fsync before any answer, flight trailers, `Unavailable` → re-attest and reseal

What the bank sees, what only the enclave sees

bank (front, store, logs)	only inside the enclave
pk_A, all `Attest`; pk_A, C_A, sig_A, pk_B, C_B, sig_B, m	v, b_A, r_A, r_B, old C_A, C_B
the sealed blobs as ciphertext; the channel pk	σ_A, σ_B (links pk to an earlier entry)
n, the result, the sides (user or bank)	next serials pk^+ and next r^+
source address of every connection (quota, memory only)	which of E1 to E4 failed

Neither side ever sees a seed or an sk_i. A reader of enclave memory learns the pending channel keys and the openings sealed to them, never an owner key.

What is persisted where
`wallet.json` per user: seed, bank keys, index, signed state, `Pending` (saved before every call that exposes a serial)
`request.json` `offer.json` `receipt.json` handed between users
`log.jsonl` public log: header, one JSON `LogEntry` per settled operation, one trailer per flight; replayed by `flight::replay` at start, a torn final flight is truncated, damage before it is fatal
`issuance.json` bank private: one line per `tv-bank issue` (time, amount, payee serial, m)
`enclave.pk` registered result key; `state.key` `issuance.key` `bank-wallet.json` `bank-genesis.json`
tv-enclave: nothing on disk; a crash loses only pending channel keys